

Komunikasi Aplikasi Perangkat Bergerak dalam Jaringan

PBP

Pengertian

- Komunikasi aplikasi perangkat bergerak
 - Proses pertukaran data antara smartphone/tablet dengan komponen lain di jaringan, seperti server, perangkat IoT, cloud, atau perangkat mobile lain.
- Komunikasi:
 - melalui Internet (Wi-Fi, mobile data)
 - melalui Bluetooth, NFC, atau Wi-Fi Direct.
- Tujuan:
 - memastikan data dapat dikirim, diproses, dan diterima secara cepat, aman, dan andal pada lingkungan yang sangat dinamis.

Arsitektur Komunikasi

- Client-Server
 - Client (mobile app) mengirim request
 - Server memproses dan mengembalikan response
- WebSocket
 - Komunikasi real-time dua arah
- Publish-Subscribe
 - Memanfaatkan broker untuk broadcast berdasarkan topik

Protokol Komunikasi

- HTTP/HTTPS (REST API/ SOAP API)
 - Standar untuk aplikasi mobile yang terkoneksi internet
 - HTTPS lebih aman karena terenkripsi TLS
- MQTT
 - Lightweight
 - Cocok untuk jaringan tidak stabil
 - Dipakai untuk komunikasi sensor–server atau sensor–mobile.
- Bluetooth/NFC
 - Komunikasi jarak dekat
 - Digunakan untuk pairing device

Mekanisme Pengiriman Data

- Synchronous
 - Aplikasi menunggu server merespons
 - Cocok untuk transaksi yang membutuhkan availability dari device
- Asynchronous
 - Aplikasi tidak harus menunggu server
 - Cocok untuk push notification, sinkronisasi data di background

Permasalahan

- Koneksi Tidak Stabil
 - Mobile device sering berpindah jaringan (Wi-Fi → 4G → 5G)
 - Solusi: retry mechanism, caching, offline-first design
- Konsumsi Energi
 - Komunikasi jaringan memakan baterai
 - Solusi: batching request, MQTT atau WebSocket untuk efisiensi data
- Keamanan
 - Mobile rentan terhadap MITM, sniffing, dan penyalahgunaan API
 - Solusi: HTTPS, token-based authentication, JWT, OAuth2, rate limiting

Contoh Studi Kasus

- Aplikasi Chat Sederhana
 - Mobile FE:
 - Halaman Login
 - Input nama
 - Halaman Chat Room
 - Menampilkan list pesan (chat bubble)
 - Input teks chat dan tombol kirim
 - Web BE:
 - Firebase Firestore (real-time)
 - Framework:
 - React Native
 - Tools:
 - React Native CLI

1. Setup Project

- Pastikan Tersedia
 - Node.js
 - Java Development Kit (JDK)
 - Android SDK
 - PATH: ANDROID_HOME, platform-tools, tools
- Buat Project
 - `npx @react-native-community/cli init ChatApp`
- Test Run
 - `npx react-native run-android`

2. Install Dependensi

- Package
 - Firebase JS SDK
 - React Navigation
 - Listener real-time Firestore
- Jalankan
 - `npm install @react-navigation/native @react-navigation/native-stack`
 - `npm install firebase`
 - `npm install react-native-screens react-native-safe-area-context`

3. Setup Firebase

- Buka <https://console.firebase.google.com>
- Buat project → "ChatApp"
- Tambahkan platform: Android
- Isi:
 - Package name: com.chatapp
(Pastikan sama dengan android/app/src/main/AndroidManifest.xml)
- Download file google-services.json
- Masukkan ke folder:
 - ChatApp/android/app/google-services.json
- Enable Firebase Authentication

3. Setup Firebase

- Install Firebase Android SDK

- Edit file: android/build.gradle

```
buildscript {  
    dependencies {  
        classpath("com.google.gms:google-services:4.3.15")  
    }  
}
```

- Edit file: android/app/build.gradle

```
apply plugin: "com.google.gms.google-services"
```

4. Setup Aplikasi

- firebase.ts
- ChatApp/screens
 - LoginScreen.tsx
 - ChatScreen.tsx

4. Setup Aplikasi (firebase.ts)

```
import { initializeApp } from "firebase/app";
import {
  getFirestore,
  collection,
  addDoc,
  serverTimestamp,
  query,
  orderBy,
  onSnapshot,
  CollectionReference,
  DocumentData,
} from "firebase/firestore";

import {
  getAuth,
  signInAnonymously,
  onAuthStateChanged,
} from "firebase/auth";
```

```
const firebaseConfig = {
  apiKey: "API_KEY",
  authDomain: "AUTH_DOMAIN",
  projectId: "PROJECT_ID",
  storageBucket: "STORAGE_BUCKET",
  messagingSenderId: "SENDER_ID",
  appId: "APP_ID"
};

const app = initializeApp(firebaseConfig);

const auth = getAuth(app);
const db = getFirestore(app);
```

```
export const messagesCollection =
  collection(db, "messages") as
  CollectionReference<DocumentData>;

export {
  auth,
  db,
  collection,
  addDoc,
  serverTimestamp,
  query,
  orderBy,
  onSnapshot,
  signInAnonymously,
  onAuthStateChanged,
};
```

4. Setup Aplikasi (App.tsx)

```
import React, { useEffect, useState } from
"react";
import { NavigationContainer } from
"@react-navigation/native";
import { createNativeStackNavigator } from
"@react-navigation/native-stack";
import LoginScreen from
"./screens/LoginScreen";
import ChatScreen from
"./screens/ChatScreen";
import { auth, signInAnonymously,
onAuthStateChanged } from "./firebase";
import { User } from "firebase/auth";

export type RootStackParamList = {
  Login: undefined;
  Chat: { name: string };
};
```

```
const Stack =
createNativeStackNavigator<RootStackParamList>();

export default function App() {
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    signInAnonymously(auth).catch(console.error);

    const unsub = onAuthStateChanged(auth, (u) => {
      setUser(u);
    });

    return () => unsub();
  }, []);

  if (!user) return null;
```

```
return (
  <NavigationContainer>
    <Stack.Navigator>
      <Stack.Screen name="Login"
component={LoginScreen} />
      <Stack.Screen name="Chat"
component={ChatScreen} />
    </Stack.Navigator>
  </NavigationContainer>
);
}
```

4. Setup Aplikasi (LoginScreen.tsx)

```
import React, { useState } from
"react";
import { View, Text, TextInput,
Button, StyleSheet } from
"react-native";
import {
NativeStackScreenProps }
from "@react-
navigation/native-stack";
import { RootStackParamList }
from "../App";
```

```
type Props =
NativeStackScreenProps<Root
StackParamList, "Login">;
```

```
export default function LoginScreen({ navigation }: Props) {
  const [name, setName] = useState<string>("");

  const handleLogin = () => {
    if (!name.trim()) return;
    navigation.navigate("Chat", { name });
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Masukkan Nama</Text>
      <TextInput
        style={styles.input}
        placeholder="Nama kamu"
        value={name}
        onChangeText={setName}
      />
      <Button title="Masuk Chat" onPress={handleLogin} />
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: { flex: 1, justifyContent:
"center", padding: 20 },
  title: { fontSize: 26, textAlign:
"center", marginBottom: 20 },
  input: {
    borderWidth: 1,
    padding: 12,
    borderRadius: 8,
    marginBottom: 20,
  },
});
```

4. Setup Aplikasi (ChatScreen.tsx)

```
import React, { useEffect, useState } from "react";
import {
  View,
  Text,
  TextInput,
  Button,
  FlatList,
  StyleSheet,
} from "react-native";

import {
  addDoc,
  serverTimestamp,
  query,
  orderBy,
  onSnapshot,
} from "../firebase";
import { messagesCollection } from "../firebase";
import { NativeStackScreenProps } from "@react-navigation/native-stack";
import { RootStackParamList } from "../App";
```

```
type MessageType = {
  id: string;
  text: string;
  user: string;
  createdAt: { seconds: number; nanoseconds: number } | null;
};

type Props = NativeStackScreenProps<RootStackParamList,
"Chat">;

export default function ChatScreen({ route }: Props) {
  const { name } = route.params;

  const [message, setMessage] = useState<string>("");
  const [messages, setMessages] =
    useState<MessageType[]>([]);
```

4. Setup Aplikasi (ChatScreen.tsx)

```
useEffect(() => {
  const q = query(messagesCollection, orderBy("createdAt", "asc"));

  const unsub = onSnapshot(q, (snapshot) => {
    const list: MessageType[] = [];
    snapshot.forEach((doc) => {
      list.push({
        id: doc.id,
        ...(doc.data() as Omit<MessageType, "id">),
      });
    });

    setMessages(list);
  });

  return () => unsub();
}, []);

const sendMessage = async () => {
  if (!message.trim()) return;
```

```
  await addDoc(messagesCollection, {
    text: message,
    user: name,
    createdAt: serverTimestamp(),
  });

  setMessage("");
};

const renderItem = ({ item }: { item: MessageType }) => (
  <View
    style={[
      styles.msgBox,
      item.user === name ? styles.myMsg : styles.otherMsg,
    ]}
  >
    <Text style={styles.sender}>{item.user}</Text>
    <Text>{item.text}</Text>
  </View>
);
```

4. Setup Aplikasi (ChatScreen.tsx)

```
return (  
  <View style={{ flex: 1 }}>  
    <FlatList  
      data={messages}  
      keyExtractor={(item) => item.id}  
      renderItem={renderItem}  
      contentContainerStyle={{ padding: 10 }}  
    />  
  
    <View style={styles.inputRow}>  
      <TextInput  
        style={styles.input}  
        placeholder="Ketik pesan..."  
        value={message}  
        onChangeText={setMessage}  
      />  
      <Button title="Kirim" onPress={sendMessage} />  
    </View>  
  </View>  
);  
}
```

```
const styles = StyleSheet.create({  
  msgBox: {  
    padding: 10,  
    marginVertical: 6,  
    borderRadius: 6,  
    maxWidth: "80%",  
  },  
  myMsg: {  
    backgroundColor: "#d1f0ff",  
    alignSelf: "flex-end",  
  },  
  otherMsg: {  
    backgroundColor: "#eee",  
    alignSelf: "flex-start",  
  },  
  sender: {  
    fontWeight: "bold",  
    marginBottom: 2,  
    fontSize: 12,  
  },  
});
```

```
inputRow: {  
  flexDirection: "row",  
  padding: 10,  
  borderTopWidth: 1,  
  borderColor: "#ccc",  
},  
input: {  
  flex: 1,  
  borderWidth: 1,  
  marginRight: 10,  
  padding: 8,  
  borderRadius: 6,  
},  
});
```

Tugas

- Buat ulang dan pelajari Aplikasi Chat Sederhana (tanya jawab di kelas)
- Kembangkan fitur: (dikumpulkan pekan depan)
 - Autentikasi pengguna menggunakan username-password
 - Auto-login
 - Simpan history chat di local storage (offline mode)
 - Upload gambar